Bridging the Gap: A Modular, Interactive Learning Platform for ROS-Based Robotics Education

Malte Matthey[1], Max Humbert[1], Florian Beyer[1], Demian Frister[2]

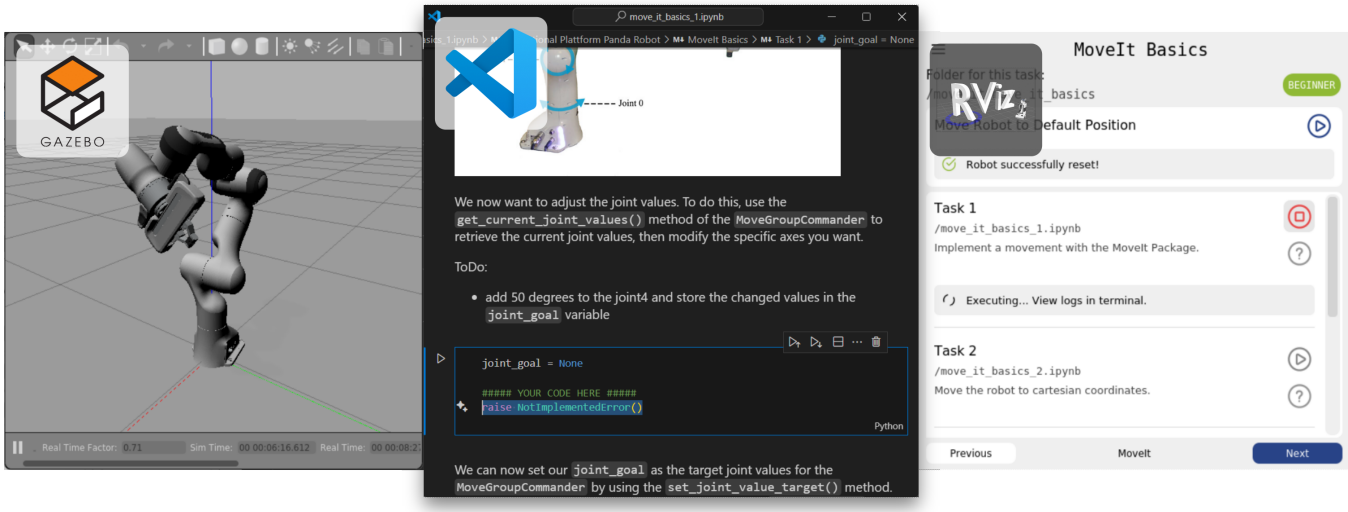

Fig. 1: Overview of the platform: Visual Studio Code with Jupyter notebooks (middle) supports interactive coding, the custom RViz plugin (right) allows managing, executing and evaluating tasks, and Gazebo (left) runs a 3D simulation.

***Abstract*— Robotics education often struggles to bridge the gap between theory and practice. In this paper, a modular and extensible learning platform is presented that integrates ROS-based robotic systems with a 3D simulator and interactive development tools. Its design enables the creation of interactive, automatically evaluated tutorials for any ROS-based robot, blending executable code with theoretical content via Jupyter Notebooks. A custom RViz plugin offers dynamic visualization to enhance the learning experience, while integration with Visual Studio Code ensures a consistent local development environment. By linking conceptual understanding with practical application, the framework provides a reproducible approach to advancing robotics education.**

I. INTRODUCTION

The ongoing technological progress and the increasing use of robotics in, for example, industrial automation and medical technology, illustrate their growing social importance [1]. In this context, there is an increasing need for effective learning platforms that provide students with a practical and motivating introduction to robotics.

A central component of modern robotic systems is the Robot Operating System (ROS). ROS provides an open, flexible middleware [2] that simplifies the development and control of complex robotic applications. The modular architecture of ROS facilitates the integration of various sensors and actuators and supports the development of complex applications. In this work, the project was developed for ROS1 to leverage the established middleware framework in a teaching context.

Many of the available tutorials, such as those found in the official ROS documentation [3], take a predominantly theoretical approach to robot programming. Topics in robotics are often introduced only superficially, without directly addressing the practical challenges of robotics. Moreover, the existing interfaces for robot programming are frequently too complex, unnecessarily hindering the learning process. However, project-based approaches and interactive learning modules provide significant added value for understanding complex robotic systems [4], [5].

The present work focuses on developing a platform for the creation and evaluation of tasks for ROS-based robots. The platform provides practice-oriented tasks designed to support the development of programming skills and to illustrate key robotics concepts. Its interface is designed for intuitive use, easing the initial engagement with robotics, while its modular structure allows for adaptation to various teaching methods and application scenarios. Additionally, the platform includes a range of tasks that cover specific topics in robotics, thereby offering learners targeted insights into the functionalities and applications of modern robotic systems.

In general, the platform presented here is intended to bridge the gap between theoretical knowledge transfer and

[1]Malte Matthey, Max Humbert, and Florian Beyer are with the Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany. They are affiliated with both the Faculty of Informatics and the Faculty of Economics and Management. Emails: [ultck, uqwfa, uninh]@student.kit.edu.

[2]Demian Frister is with the AIFB BIS at the Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany. Email: demian.frister@kit.edu

practical application, thus making a lasting contribution to the advancement of robotics education.

II. RELATED WORK

There is already a lot of learning material for ROS, including the tutorials in the official ROS documentation [3]. The same applies to other topics in robotics, such as MoveIt [6], which is an open source software for motion planning, manipulation, and collision detection in ROS-based robotic systems. The problem with the official documentation is that topics are often addressed purely theoretically, while practical tasks and applications are neglected, leading to lower motivation among learners [5]. This issue is to be addressed in the present work.

Several studies in recent years have designed various learning platforms for ROS. For example, there are project-based approaches [4], [7] that provide students with an introduction to ROS. Although these approaches offer valuable fundamentals, they often rely on conventional teaching methods and employ outdated concepts, such as the exclusive use of physical robotic systems. Moreover, these studies frequently use proprietary platforms that restrict access to extensive topics such as MoveIt and do not offer the ability to flexibly organize instruction through interactive notebooks.The present work fills this gap with a modular approach that uses Jupyter Notebooks [8] to cover a wide range of topics, from ROS fundamentals and the application of MoveIt to other modules relevant to practice, while allowing individual customization of the learning process. This approach is further enhanced by integrating RViz and VSCode [9], which links theoretical concepts directly to practical applications and allows continuous evaluation of learning progress.

In addition, there are existing physical learning platforms, such as Lego Mindstorms [10] and Robobo [11], which are often associated with high acquisition costs, thus limiting their practical use in educational institutions.

RViz has established itself as a central, interactive visualization tool in robotics, used not only for displaying sensor data and robot states but also as a debugging and teaching tool. It has been demonstrated [12], [13] that the use of RViz can reduce the complexity of visual representations and support the learning process. In this work, RViz serves two purposes: It provides a virtual representation of ROS data and is extended with a custom RViz plugin to enhance the interactive learning experience. This integration enables learners to immediately connect theory and practice.

In summary, the presented work offers a novel approach that builds on the strengths of existing learning platforms while specifically addressing their weaknesses, particularly in terms of flexibility and didactic preparation. The integration of modern tools such as interactive notebooks and the extended use of RViz provides a solid foundation for future teaching and learning concepts in ROS-based robotics.w

III. METHODOLOGY

The proposed educational platform consists of two key elements: the simulated environment, which includes the simulation software and the robot, and the learning platform implemented as an RViz [14] plugin. Communication between these elements is achieved using standard ROS trajectory messages, a protocol widely used in robotic applications. Although related projects [**ventuzelos˙teaching˙2022-1**] used the Flatland simulator [15], Gazebo was selected in our example due to its ROS compatibility and simplified 3D environment for robotic simulations. However, the platform is not restricted to Gazebo; any ROS-compatible simulator can be used.

To support the development of the educational platform, the Franka Emika Panda Robot [16] was chosen as an example due to its native ROS support and high compatibility with the Gazebo simulator. Importantly, this choice was made solely for demonstration purposes, and the platform is designed to support tutorials for any ROS-based robot. Currently, launch files are available to start the simulator and the robot in the correct configuration. These launch files are designed with flexibility in mind, allowing easy adaptation for alternative robots with minimal modifications.

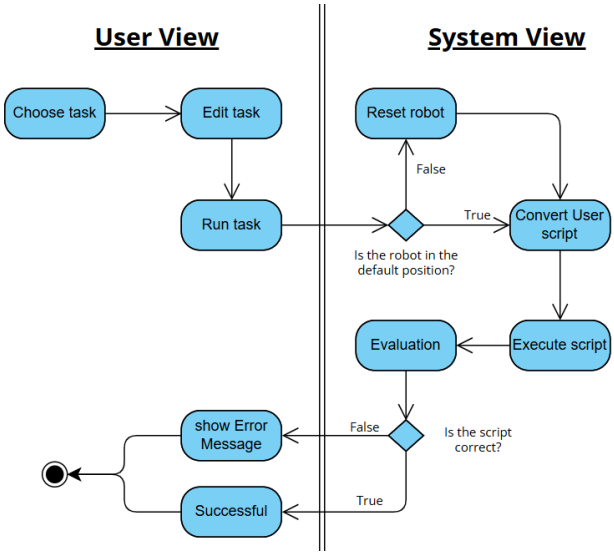

Fig. 3: Abstract process of executing a task

To guide users through the learning process, a step-by-step guide has been developed, where tasks build progressively on one another. After selecting a task, a Jupyter notebook [8] is provided for solving the problem. The division into markdown and code cells within a Jupyter notebook enables the combination of executable code and theoretical knowledge in a single document, along with images, HTML, LaTeX, and more. Additionally, Visual Studio Code [9] is provided as an integrated development environment (IDE), further enhancing the learning process by offering a robust environment for code development and debugging.

The definition and management of tasks are handled through a centralized JSON file. This not only provides a high degree of modularity when creating additional tasks or adding new robots, but also allows to make task-specific decisions for script execution and evaluation.

The fields that the task must have in the JSON file are shown in Table I, those for subtasks are shown in Table II. Additionally, the JSON is expandable, and custom fields can be added .

TABLE I: Fields for a **task** in the JSON file

Name	Description	Optional
Title	The title of the task.	
Folder Name	The folder in which the task is located.	
Topic	The topic to which the task belongs.	
Difficulty Level	The appropriate difficulty level for the task.	
Subtasks	A list of subtasks that implement the task (at least one).	
Execute Previous Subtasks	Executes all previous subtasks before executing the current one (used when subtasks are related).	X

TABLE II: Fields for a **subtask** in the JSON file

Name	Description	Optional
Title	The title of the subtask.	
Description	A brief description of the subtask.	
Solution and Evaluation Files	The names of the corresponding solution and evaluation files.	
Evaluation Mode	Defines whether the evaluation is executed after or in parallel with the script.	X
Reset Robot	Enables or disables the automatic reset of the robot before executing the script.	X
Timeout Duration	Sets the timeout duration for the subtask execution.	X

Once defined, tasks are dynamically rendered in the RViz plugin frontend. This integration allows users to execute tasks directly through the graphical interface.

Before execution, the code contained in the Jupyter notebooks is automatically converted into a standard Python script, ensuring that the content remains unchanged. The converted Python file is then executed. Since the user programs in Python within the notebook and communicates with ROS via the ROS Python API, it is essential that the selected robot is compatible with ROS. Although a native Python API of the robot is not strictly necessary, its presence allows the development of learning tasks directly through this interface. Another reason why the Panda robot was chosen as an example robot is that it has such a Python API.

After a task is executed, the system provides immediate feedback by displaying the evaluation results, whether the task was completed successfully, encountered errors, or the desired objectives were not achieved. To effectively communicate the execution state of each script and potential success or error messages, a design inspired by the state display of individual Jupyter cells in Google Colab [17] was adopted. This approach provides clear, organized visual feedback, enabling users to monitor progress and diagnose issues in a familiar, intuitive manner. Unlike other projects that may rely on external tools for evaluation, the presented approach encapsulates the evaluation logic within separate Python files, making it possible to display the feedback in the frontend.

The system is deployed in a containerized environment to ensure reproducibility and isolation. Thanks to its modular design - in particular the JSON file for tasks - and its containerisation, the project encourages users to develop for the learning platform themselves by adding their own tasks or integrate new robots without having to develop core components. In addition, project-based learning platforms offer a higher learning effect than just reading documentation [18].

IV. IMPLEMENTATION

The system's containerization is realized through Docker. DevContainers [19] enable development in VS Code [9] with a preconfigured environment that includes all dependencies, ensuring consistency across different machines. A full local setup is also supported.

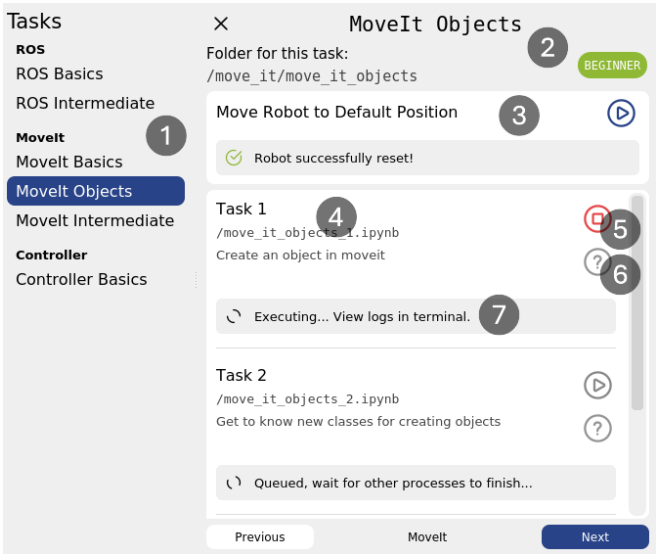

Fig. 4: User interface screenshot.

TABLE III: Core UI elements and their functions

Nr.	Feature	Description
1	Sidebar	Lists tasks by topic and allows selection.
2	Task Header	Displays the task title, difficulty, and folder path.
3	Reset Robot	Manually resets the robot to its default state.
4	Subtask Info	Shows the subtask title, description, and associated Jupyter Notebook file.
5	Start / Stop	Controls user or (if selected) solution script execution.
6	Help	Opens a submenu to toggle solution visibility or reset the notebook.
7	Execution Info	Displays execution progress, success, or error messages.

The **User Interface** (UI) is implemented as an RViz plugin. An RViz plugin was chosen to seamlessly integrate with existing ROS visualization tools and provide an interactive environment for users. It was developed using the Qt Framework[1] (Qt5 [20]), ensuring full compatibility with RViz [14], which is built on Qt5. The plugin provides full functionality through its user interface. Figure 4 shows the interface, and Table III summarizes the core UI elements and their respective functions.

Besides the UI module, the system includes a **Task Management** module. This module organizes tasks hierarchically. Each topic comprises multiple tasks. Each task is subdivided into several subtasks. Subtasks consist of a Jupyter Notebook containing a placeholder solution and an evaluation script for validation. Task and subtask definitions are stored in a JSON configuration file for dynamic loading of tasks. This file also defines parameters for different execution modes, as described in Section III. Since each task already contains a solution, the notebooks require preprocessing. The system scans each notebook for code cells marked as solutions and replaces them with a `NotImplementedError` placeholder. This ensures that users provide their own implementations. Solution cells can be toggled dynamically at runtime by the user.

A third module handles **Script Execution**. Jupyter Notebooks are converted into Python scripts and executed according to the specifications in the JSON configuration file. Error handling is included. Errors are reported if a script has syntax or runtime issues or if evaluation fails due to an incorrect solution. Errors are displayed in the UI and logged for reference.

The full source code of the project is available at `https://gitlab.kit.edu/kit/aifb/BIS/kit-bis/robotik/ros-learning-platform`.

V. CONCLUSIONS

In conclusion, this work successfully revisits its primary objective of bridging the gap between theoretical knowledge and practical application in robotics education. By developing a modular, ROS-based learning platform, the study has demonstrated how interactive tools such as Jupyter Notebooks and an enhanced RViz plugin can transform the learning experience, making complex concepts more accessible and engaging.

The platform is demonstrated using the Franka Emika Panda robot, serving as a concrete example to illustrate its capabilities. This approach lays the groundwork for the development of additional tutorials and tasks tailored to a broader range of robotic systems, thereby enriching the educational experience over time. Although a formal evaluation was not conducted, preliminary observations suggest that the system has the potential to enhance usability and student engagement significantly.

[1]Qt is a full development framework with tools designed to streamline the creation of applications and user interfaces for desktop, embedded, and mobile platforms.

It is important to note that a significant limitation of this work is the exclusive use of ROS1, even though ROS2 has become the current standard in robotics. The development of a ROS2 version of the learning platform is still pending.

Looking ahead, there is an opportunity to further refine the learning platform by developing a dedicated Visual Studio Code plugin alongside the existing RViz plugin, offering a more streamlined learning experience. Additionally, the project's reliance on DevContainers paves the way for integration with cloud-based solutions [21] for the learning platform. This approach would reduce the need for high-performance local hardware, which is a major benefit for students without access to powerful computing resources. As a result, advanced robotic simulations and practical exercises would become more accessible to a broader audience.

Overall, the work provides a solid and extensible foundation for future advancements in robotics education, emphasizing both scalability and accessibility as key components of its long-term vision.

ACKNOWLEDGMENT

This work was conducted as part of a seminar at the Institute of Applied Informatics and Formal Description Methods (AIFB), Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany. We thank our seminar supervisor, Demian Frister, M.Sc., for his support during the seminar.

REFERENCES

[1] J. López-Belmonte, A. Segura Robles, A. Moreno Guerrero, and M. Parra Gonzalez, "Robotics in Education: A Scientific Mapping of the Literature in Web of Science", *Electronics*, vol. 10, pp. 1–18, Jan. 2021. DOI: `10.3390/electronics10030291`.

[2] M. Quigley, K. Conley, B. Gerkey, *et al.*, *ROS: an open-source Robot Operating System*. Jan. 2009, vol. 3, Journal Abbreviation: ICRA Workshop on Open Source Software Publication Title: ICRA Workshop on Open Source Software.

[3] *ROS/Tutorials - ROS Wiki*. [Online]. Available: `https://wiki.ros.org/ROS/Tutorials` (visited on 02/21/2025).

[4] Y. Wang, Z. Zhang, and Y. Chang, "A project-based platform for students' Robot Operation System (ROS) programming experience", Jun. 2022. DOI: `10.18260/1-241272`.

[5] P. Blumenfeld, E. Soloway, R. Marx, and J. Krajcik, "Motivating Project-Based Learning: Sustaining the Doing, Supporting the Learning", *Educational Psychologist*, vol. 26, pp. 369–398, Nov. 2011. DOI: `10.1207/s15326985ep2603&4_8`.

[6] *MoveIt Tutorials — moveit_tutorials Noetic documentation*. [Online]. Available: `https://moveit.github.io/moveit_tutorials/` (visited on 02/21/2025).

[7] V. E. M. Ventuzelos, "Teaching Reinforcement Learning in ROS 1/2 using a Mobile Robot and Its Simulation", 2022.

[8] *Project Jupyter Documentation — Jupyter Documentation 4.1.1 alpha documentation*. [Online]. Available: https://docs.jupyter.org/en/latest/ (visited on 02/21/2025).

[9] *Documentation for Visual Studio Code*. [Online]. Available: https://code.visualstudio.com/docs (visited on 02/15/2025).

[10] *LEGO® MINDSTORMS®— About — Official LEGO® Shop US*, en. [Online]. Available: https://www.lego.com/en-us/themes/mindstorms/about (visited on 02/21/2025).

[11] F. Bellas, M. Naya, G. Varela, *et al.*, "Robobo: The Next Generation of Educational Robot", in Jan. 2018, pp. 359–369, ISBN: 978-3-319-70835-5. DOI: 10.1007/978-3-319-70836-2_30.

[12] B. Ikeda and D. Szafir, "Advancing the Design of Visual Debugging Tools for Roboticists", in *2022 17th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, 2022, pp. 195–204. DOI: 10.1109/HRI53351.2022.9889392.

[13] C. Gomez, A. Hernández, J. Crespo, and R. Barber, "LEARNING ROBOTICS THROUGH A FRIENDLY GRAPHICAL USER INTERFACE", Nov. 2016, pp. 483–492. DOI: 10.21125/iceri.2016.1120.

[14] H. Kam, S.-H. Lee, T. Park, and C.-H. Kim, "RViz: A toolkit for real domain data visualization", *Telecommunication Systems*, vol. 60, pp. 1–9, Oct. 2015. DOI: 10.1007/s11235-015-0034-5.

[15] *Welcome to Flatland's documentation! — Flatland documentation*. [Online]. Available: https://flatland-simulator.readthedocs.io/en/latest/ (visited on 02/17/2025).

[16] *Franka Emika Panda robot - RoboDK*. [Online]. Available: https://robodk.com/robot/de/Franka/Emika-Panda (visited on 02/17/2025).

[17] T. Carneiro, R. V. Medeiros Da NóBrega, T. Nepomuceno, G.-B. Bian, V. H. C. De Albuquerque, and P. P. R. Filho, "Performance Analysis of Google Colaboratory as a Tool for Accelerating Deep Learning Applications", *IEEE Access*, vol. 6, pp. 61 677–61 685, 2018. DOI: 10.1109/ACCESS.2018.2874767.

[18] L. Zhang and Y. Ma, "A study of the impact of project-based learning on student learning effects: A meta-analysis study", *Frontiers in Psychology*, vol. 14, Jul. 2023. DOI: 10.3389/fpsyg.2023.1202728.

[19] *Developing inside a Container*, en. [Online]. Available: https://code.visualstudio.com/docs/devcontainers/containers (visited on 02/15/2025).

[20] *Qt 5.15*. [Online]. Available: https://doc.qt.io/qt-5/ (visited on 02/15/2025).

[21] *Coder — Cloud Development Environment: Remote & Self Hosted*, en. [Online]. Available: https://coder.com/ (visited on 02/22/2025).